Tries and Bloom filters

Searching by prefix, and answering "maybe" with very little memory

Mohammed Said BELAID
said.belaid@univ-usto.dz
github.com/msbelaid

28/10/2026



Outline

1 The problem: searching by prefix
2 Tries
3 Operations on a trie
4 Cost of a trie
5 Tries in practice
6 Probabilistic data structures
7 Bloom filters
8 How wrong can a Bloom filter be?
9 Bloom filters in practice
10 Summary



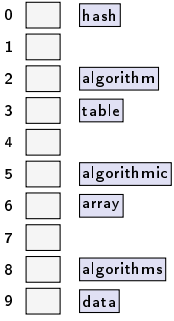
Outline

1 The problem: searching by prefix
2 Tries
3 Operations on a trie
4 Cost of a trie
5 Tries in practice
6 Probabilistic data structures
7 Bloom filters
8 How wrong can a Bloom filter be?
9 Bloom filters in practice
10 Summary



Back to the open question

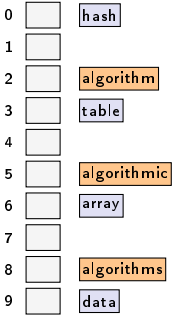
A hash table holds the words of a dictionary. Find the words starting with "algo".





Back to the open question

A hash table holds the words of a dictionary. Find the words starting with "algo".

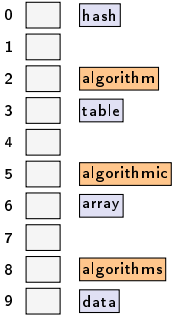


The three words are in unrelated buckets.



Back to the open question

A hash table holds the words of a dictionary. Find the words starting with "algo".



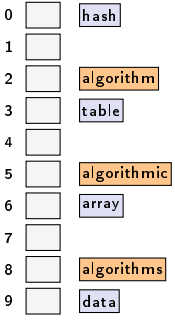
The three words are in unrelated buckets.

A good hash function scatters similar keys *on purpose*.



Back to the open question

A hash table holds the words of a dictionary. Find the words starting with "algo".



The three words are in unrelated buckets.

A good hash function scatters similar keys *on purpose*.

The only way: scan the whole table, $O(n)$.



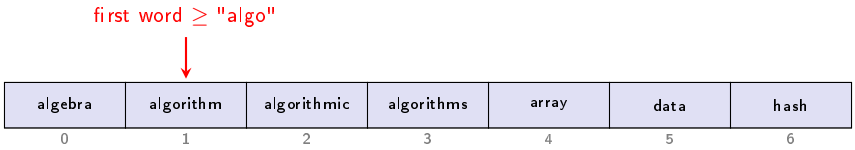
First fix: a sorted array

algebra	algorithm	algorithmic	algorithms	array	data	hash
0	1	2	3	4	5	6

- Words with the same prefix are neighbours in sorted order.



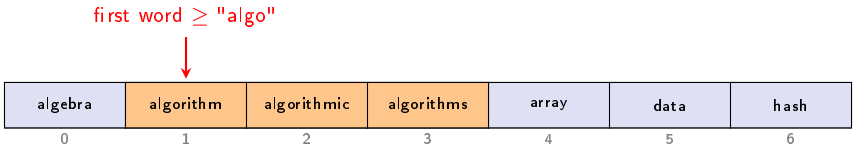
First fix: a sorted array



- Words with the same prefix are neighbours in sorted order.
- Binary search for the first word $\geq$ `"algo"`: $O(\log n)$ comparisons.



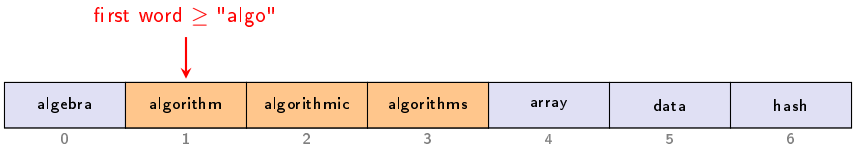
First fix: a sorted array



- Words with the same prefix are neighbours in sorted order.
- Binary search for the first word $\geq$ "algo": $O(\log n)$ comparisons.
- Then read the next words while they start with "algo".



First fix: a sorted array



- Words with the same prefix are neighbours in sorted order.
- Binary search for the first word $\geq$ "algo": $O(\log n)$ comparisons.
- Then read the next words while they start with "algo".
- But inserting a new word shifts the array: $O(n)$ (see the Dynamic Arrays course).



The idea: share the prefixes

```
car
card
care
cat
```



The idea: share the prefixes

car
card
care
cat



The idea: share the prefixes

car
card
care
cat



The idea: share the prefixes

car
card
care
cat

- Store each common prefix once.
- A word is a path of letters, from a root.
- Words that share a prefix share the beginning of their path.

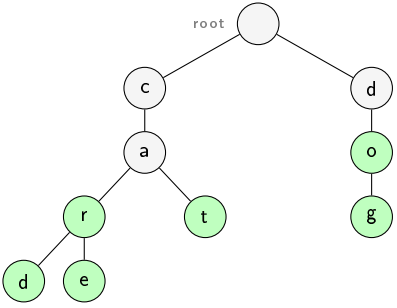


Outline

1 The problem: searching by prefix
2 Tries
3 Operations on a trie
4 Cost of a trie
5 Tries in practice
6 Probabilistic data structures
7 Bloom filters
8 How wrong can a Bloom filter be?
9 Bloom filters in practice
10 Summary



What is a trie?

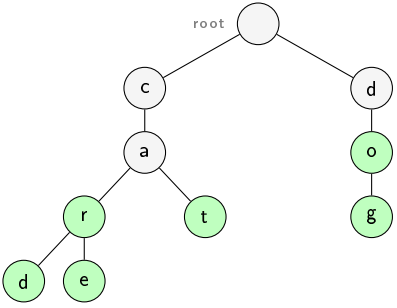


Trie (prefix tree)

A tree where each node is reached by one letter, and each path from the root spells a prefix. A node is *marked* when its path is a whole word.



What is a trie?



Trie (prefix tree)

A tree where each node is reached by one letter, and each path from the root spells a prefix. A node is *marked* when its path is a whole word.

Words: car, card, care, cat, do, dog.

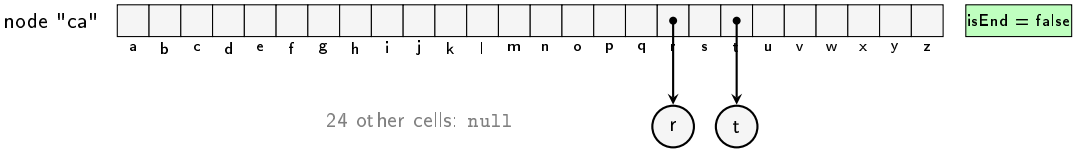
 marks the end of a word.



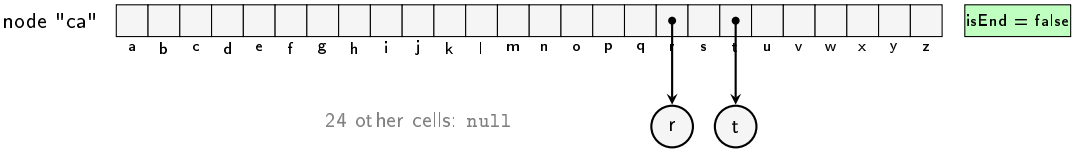
Inside a node



Inside a node



Inside a node



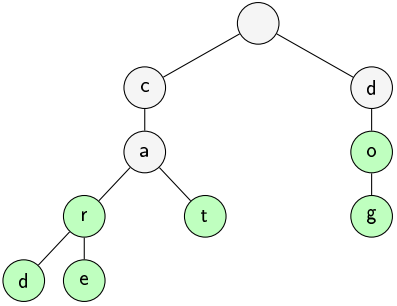
- `children[26]`: the child reached by each letter (`children[c - 'a']`).
- `isEnd`: does a word end here?
- Following a letter is a direct array access: $O(1)$.

Outline

1 The problem: searching by prefix
2 Tries
3 Operations on a trie
4 Cost of a trie
5 Tries in practice
6 Probabilistic data structures
7 Bloom filters
8 How wrong can a Bloom filter be?
9 Bloom filters in practice
10 Summary

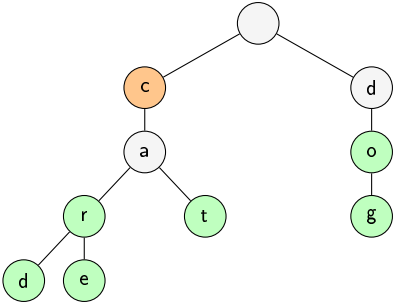


Inserting a word




```
insert("cart"):
```

Inserting a word

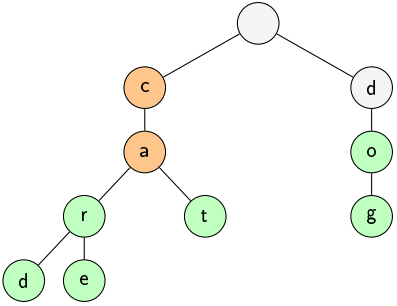


`insert("cart")`:

Follow c, a, r: these nodes already exist.



Inserting a word

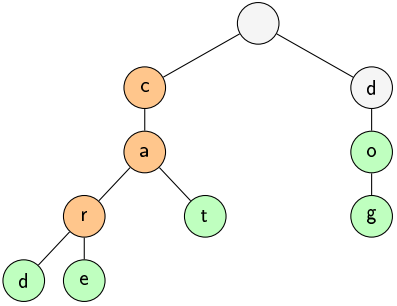


`insert("cart")`:

Follow c, a, r: these nodes already exist.



Inserting a word

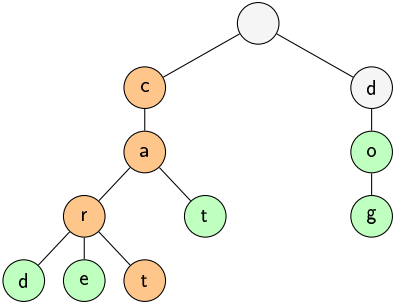


`insert("cart")`:

Follow c, a, r: these nodes already exist.



Inserting a word



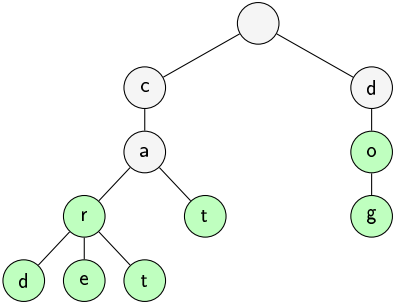
`insert("cart")`:

Follow c, a, r: these nodes already exist.

No child for t: create a new node.



Inserting a word



`insert("cart")`:

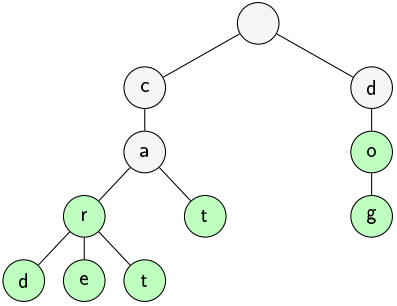
Follow c, a, r: these nodes already exist.

No child for t: create a new node.

End of the word: mark the last node.



Inserting a word



`insert("cart")`:

Follow c, a, r: these nodes already exist.

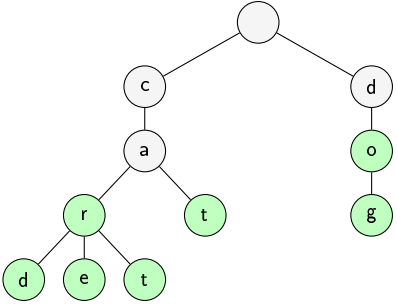
No child for t: create a new node.

End of the word: mark the last node.

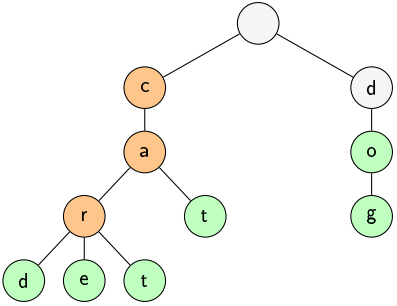
Cost: $O(m)$, with m the length of the word.



Searching a word



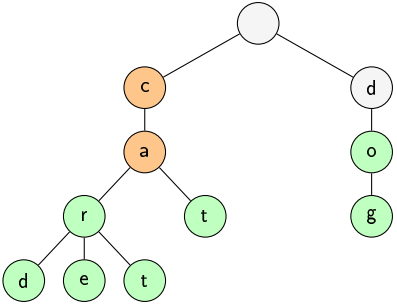
Searching a word



`contains("car")`: path c, a, r; the node is marked: **true**.



Searching a word

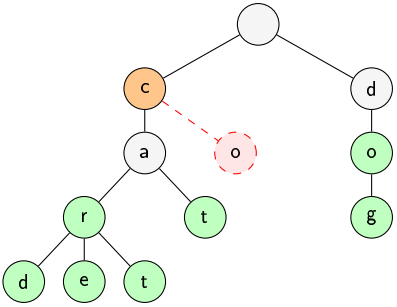


`contains("car")`: path c, a, r; the node is marked: **true**.

`contains("ca")`: path c, a; not marked: **false**.



Searching a word



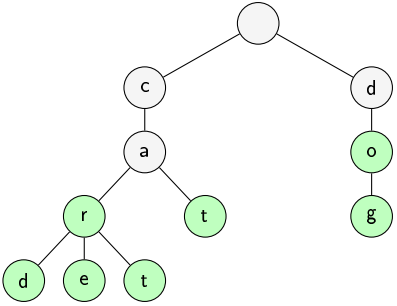
`contains("car")`: path c, a, r; the node is marked: **true**.

`contains("ca")`: path c, a; not marked: **false**.

`contains("cow")`: no child o under c: **false**.



Searching a word



`contains("car")`: path c, a, r; the node is marked: **true**.

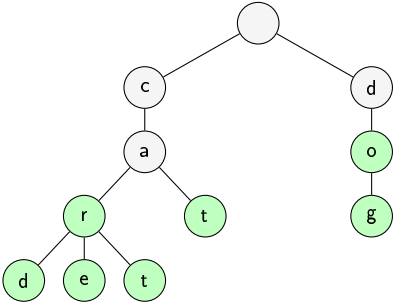
`contains("ca")`: path c, a; not marked: **false**.

`contains("cow")`: no child o under c: **false**.

`startsWith("ca")`: the path exists: **true**. The mark does not matter.



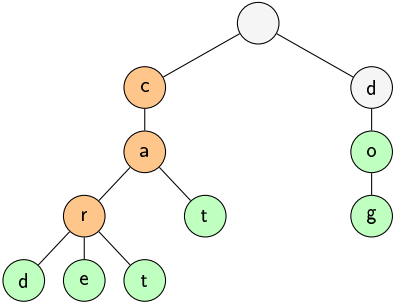
Autocomplete: all the words with a prefix



`wordsWithPrefix("car")`:



Autocomplete: all the words with a prefix

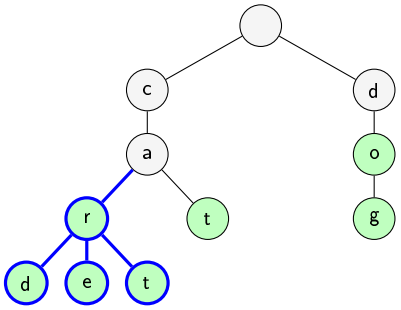


`wordsWithPrefix("car")`:

1. Walk down to the node of the prefix: $O(m)$.



Autocomplete: all the words with a prefix



`wordsWithPrefix("car")`:

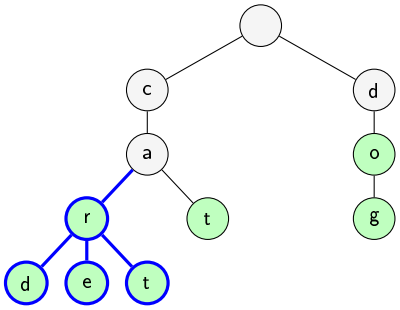
1. Walk down to the node of the prefix: $O(m)$.

2. Visit its whole subtree (depth-first), and collect every marked node:

car, card, care, cart



Autocomplete: all the words with a prefix



`wordsWithPrefix("car")`:

1. Walk down to the node of the prefix: $O(m)$.

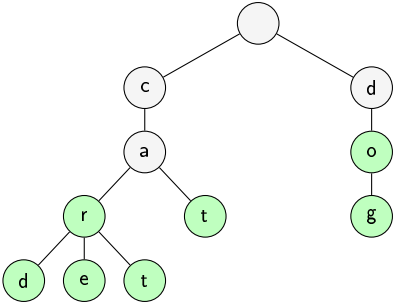
2. Visit its whole subtree (depth-first), and collect every marked node:

car, card, care, cart

Visiting the children from a to z gives the words in *alphabetical order*.

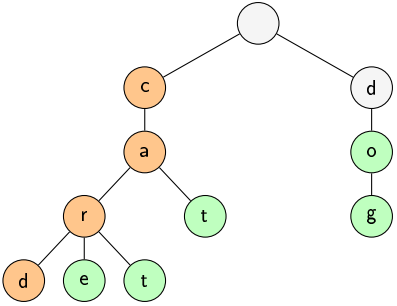


Deleting a word





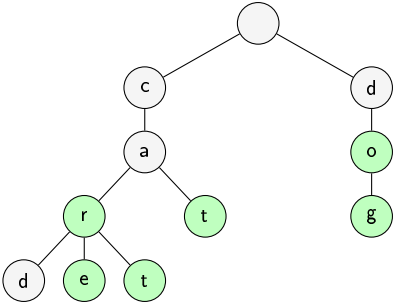
Deleting a word



`remove("card")`: find the node of the word.



Deleting a word

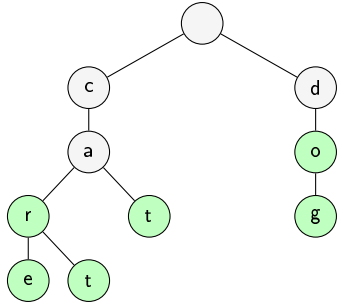


`remove("card")`: find the node of the word.

Unmark it.



Deleting a word



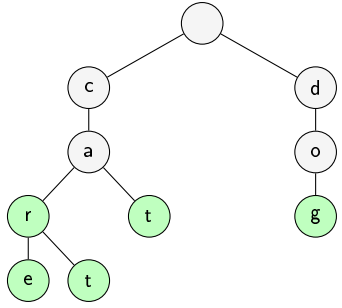
`remove("card")`: find the node of the word.

Unmark it.

It has no child and is not marked: remove it.
Its parent "car" is marked: stop.



Deleting a word



`remove("card")`: find the node of the word.

Unmark it.

It has no child and is not marked: remove it.
Its parent "car" is marked: stop.

`remove("do")`: unmark it, but keep the node: "dog" goes through it.



Exercise: build a trie

Insert the words: `to`, `tea`, `ted`, `ten`, `i`, `in`, `inn`.

1. Draw the trie. How many nodes does it have, root included?
2. Which nodes are marked?
3. List the words returned by `wordsWithPrefix("te")`, in order.
4. Remove `in`, then `inn`: draw the trie after each removal.



Outline

1 The problem: searching by prefix
2 Tries
3 Operations on a trie
4 Cost of a trie
5 Tries in practice
6 Probabilistic data structures
7 Bloom filters
8 How wrong can a Bloom filter be?
9 Bloom filters in practice
10 Summary



Time

n words, m = length of the searched word, r = number of answers.

Operation	Hash table	Sorted array	Trie
Search a word	$O(m)$ on average	$O(m \log n)$	$O(m)$
Insert a word	$O(m)$ on average	$O(n)$: shifting	$O(m)$
Words with a prefix	$O(n)$: scan all	$O(m \log n + r)$	$O(m + \text{subtree})$
Words in sorted order	sort: $O(n \log n)$	already sorted	depth-first traversal



Time

n words, m = length of the searched word, r = number of answers.

Operation	Hash table	Sorted array	Trie
Search a word	$O(m)$ on average	$O(m \log n)$	$O(m)$
Insert a word	$O(m)$ on average	$O(n)$: shifting	$O(m)$
Words with a prefix	$O(n)$: scan all	$O(m \log n + r)$	$O(m + \text{subtree})$
Words in sorted order	sort: $O(n \log n)$	already sorted	depth-first traversal

- A trie never depends on n for one word: only on its length.
- Hashing a string is $O(m)$ too: the hash table is not "free".



Outline

1 The problem: searching by prefix
2 Tries
3 Operations on a trie
4 Cost of a trie
5 Tries in practice
6 Probabilistic data structures
7 Bloom filters
8 How wrong can a Bloom filter be?
9 Bloom filters in practice
10 Summary



Where are tries used?

- Autocomplete: search bars, code editors, phone keyboards.
- Spell checking: is this word in the dictionary? Which words are close to it?
- IP routing: a router finds the longest prefix of the address in its table (a binary trie of bits).
- Word games: stop exploring as soon as the letters are not a prefix of any word (Boggle, crosswords).



Outline

1 The problem: searching by prefix
2 Tries
3 Operations on a trie
4 Cost of a trie
5 Tries in practice
6 Probabilistic data structures
7 Bloom filters
8 How wrong can a Bloom filter be?
9 Bloom filters in practice
10 Summary



A very large set

A browser checks every visited URL against a list of 1 million malicious URLs.

- Hash set: $O(1)$ per check, but it stores every URL. $\approx$ 100 MB of memory.



A very large set

A browser checks every visited URL against a list of 1 million malicious URLs.

- Hash set: $O(1)$ per check, but it stores every URL. ≈ 100 MB of memory.
- Trie: shares prefixes, but one node per character: not better.



A very large set

A browser checks every visited URL against a list of 1 million malicious URLs.

- Hash set: $O(1)$ per check, but it stores every URL. $\approx$ 100 MB of memory.
- Trie: shares prefixes, but one node per character: not better.
- Almost every URL is safe: the answer is "no" 99.9% of the time.



A very large set

A browser checks every visited URL against a list of 1 million malicious URLs.

- Hash set: $O(1)$ per check, but it stores every URL. $\approx$ 100 MB of memory.
- Trie: shares prefixes, but one node per character: not better.
- Almost every URL is safe: the answer is "no" 99.9% of the time.



A very large set

A browser checks every visited URL against a list of 1 million malicious URLs.

- Hash set: $O(1)$ per check, but it stores every URL. $\approx$ 100 MB of memory.
- Trie: shares prefixes, but one node per character: not better.
- Almost every URL is safe: the answer is "no" 99.9% of the time.

Does the answer have to be exact?



Trading exactness for memory

Probabilistic data structure

A structure that answers with a small, *controlled* probability of error, in exchange for much less memory or time than an exact structure.

- We choose the error rate: 1%, 0.1%, ...



Trading exactness for memory

Probabilistic data structure

A structure that answers with a small, *controlled* probability of error, in exchange for much less memory or time than an exact structure.

- We choose the error rate: 1%, 0.1%, ...
- The error is one-sided: we know which answers can be wrong.



Trading exactness for memory

Probabilistic data structure

A structure that answers with a small, *controlled* probability of error, in exchange for much less memory or time than an exact structure.

- We choose the error rate: 1%, 0.1%, ...
- The error is one-sided: we know which answers can be wrong.
- A "maybe" can be checked afterwards by a slow, exact method.



Randomized or approximate?

Two different uses of randomness:

	Randomized, exact	Approximate
The answer	always correct	rarely wrong
Randomness affects	the running time	the answer
Examples	randomized quicksort, skip list	Bloom filter, HyperLogLog



Randomized or approximate?

Two different uses of randomness:

	Randomized, exact	Approximate
The answer	always correct	rarely wrong
Randomness affects	the running time	the answer
Examples	randomized quicksort, skip list	Bloom filter, HyperLogLog

Hash tables already rely on "random-looking" hash values: $O(1)$ on average, but always the right answer.



A family of probabilistic structures

Structure	Question answered	Error
Bloom filter	Is x in the set?	false "yes"
Count-Min Sketch	How many times did x appear?	overestimates the count
HyperLogLog	How many distinct elements?	$\approx 1\%$ with 12 KB
MinHash	How similar are two sets?	approximate similarity



A family of probabilistic structures

Structure	Question answered	Error
Bloom filter	Is x in the set?	false "yes"
Count-Min Sketch	How many times did x appear?	overestimates the count
HyperLogLog	How many distinct elements?	$\approx 1\%$ with 12 KB
MinHash	How similar are two sets?	approximate similarity

Today: the Bloom filter, the simplest and the most used.



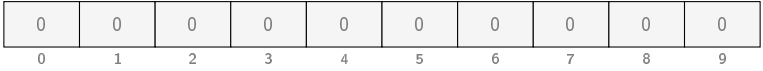
Outline

1 The problem: searching by prefix
2 Tries
3 Operations on a trie
4 Cost of a trie
5 Tries in practice
6 Probabilistic data structures
7 Bloom filters
8 How wrong can a Bloom filter be?
9 Bloom filters in practice
10 Summary



First idea: a hash set of bits

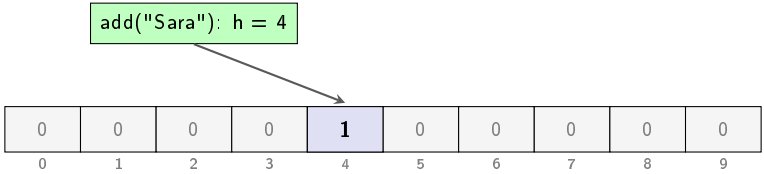
Keep only one bit per bucket: the bucket is used or not. $m = 10$ bits, one hash function h.





First idea: a hash set of bits

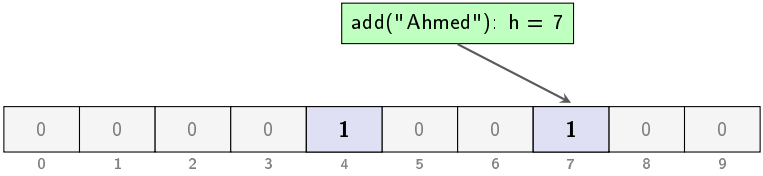
Keep only one bit per bucket: the bucket is used or not. $m = 10$ bits, one hash function h.





First idea: a hash set of bits

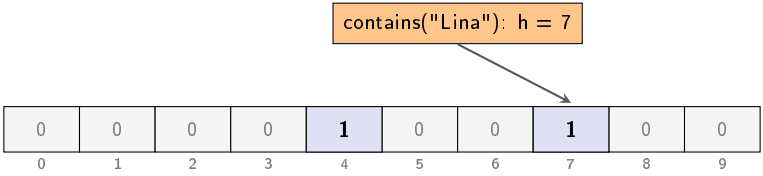
Keep only one bit per bucket: the bucket is used or not. $m = 10$ bits, one hash function h.





First idea: a hash set of bits

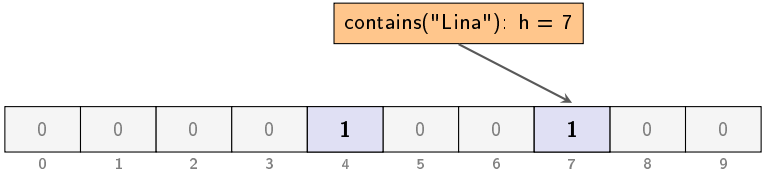
Keep only one bit per bucket: the bucket is used or not. $m = 10$ bits, one hash function h.





First idea: a hash set of bits

Keep only one bit per bucket: the bucket is used or not. $m = 10$ bits, one hash function h.



- Bit 7 is 1, but Lina was never added: a false positive, caused by a collision.
- A bit set to 0 is certain: nothing was ever added there.
- 10 bits for any number of names, but many collisions.



Bloom filter: k hash functions

Bloom filter

An array of m bits, all 0 at the start, and k hash functions $h_1, \ldots, h_k$ with values in $[0, m)$.

- `add(x)`: set the k bits $h_1(x), \ldots, h_k(x)$ to 1.
- `mightContain(x)`: are the k bits $h_1(x), \ldots, h_k(x)$ all 1?



Bloom filter: k hash functions

Bloom filter

An array of m bits, all 0 at the start, and k hash functions $h_1, \ldots, h_k$ with values in $[0, m)$.

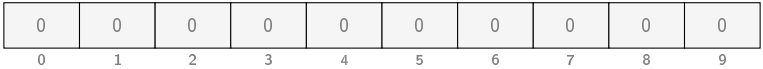
- `add(x)`: set the k bits $h_1(x), \ldots, h_k(x)$ to 1.
- `mightContain(x)`: are the k bits $h_1(x), \ldots, h_k(x)$ all 1?

- A false positive needs a collision on all k bits: much rarer.
- Both operations cost $O(k)$, whatever the number of elements.
- The elements themselves are never stored.



Inserting: an example

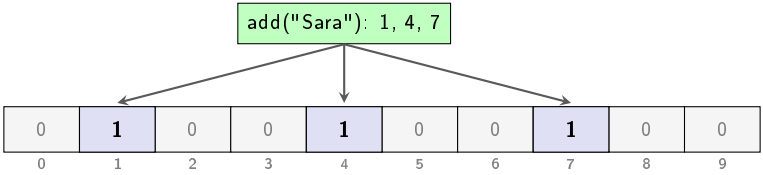
$m = 10$ bits, $k = 3$ hash functions.





Inserting: an example

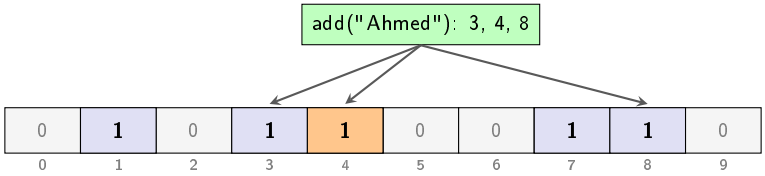
$m = 10$ bits, $k = 3$ hash functions.



- Sara sets bits 1, 4 and 7.

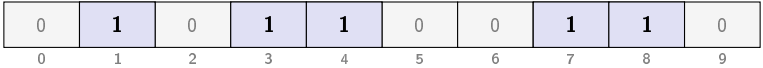
Inserting: an example

$m = 10$ bits, $k = 3$ hash functions.

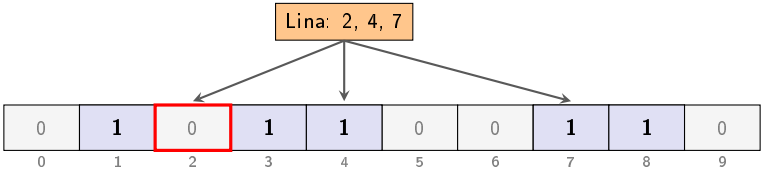


- Sara sets bits 1, 4 and 7.
- Ahmed sets bits 3, 4 and 8. Bit 4 is already 1: elements share bits.

Looking up: "no" or "maybe"

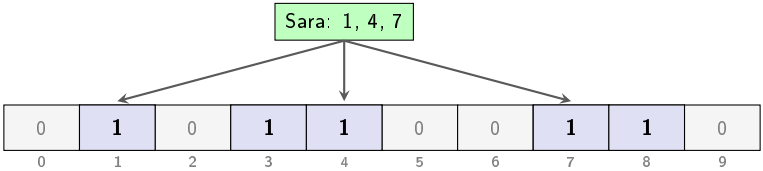


Looking up: "no" or "maybe"



- Lina: bit 2 is 0. Definitely not in the set.

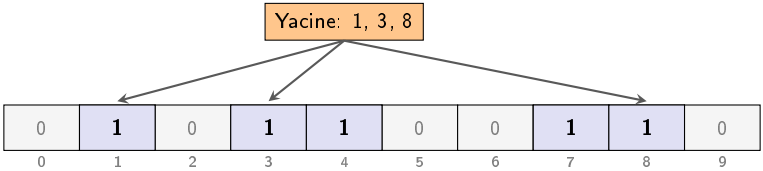
Looking up: "no" or "maybe"



- Lina: bit 2 is 0. Definitely not in the set.
- Sara: all three bits are 1. Maybe in the set (and she is).

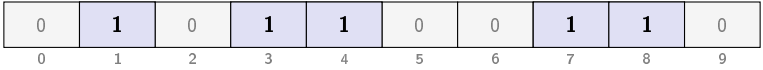


Looking up: "no" or "maybe"



- Lina: bit 2 is 0. Definitely not in the set.
- Sara: all three bits are 1. Maybe in the set (and she is).
- Yacine: all three bits are 1, set by Sara and Ahmed. Maybe, but he is not: a false positive.

Looking up: "no" or "maybe"

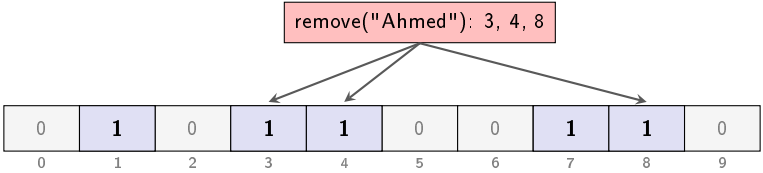


- Lina: bit 2 is 0. Definitely not in the set.
- Sara: all three bits are 1. Maybe in the set (and she is).
- Yacine: all three bits are 1, set by Sara and Ahmed. Maybe, but he is not: a false positive.

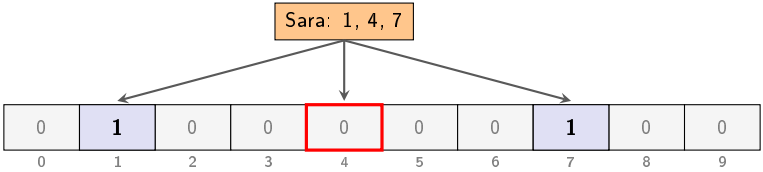
One-sided error

No is always right: no false negatives. **Yes** means *maybe*: false positives are possible.

Why we cannot delete



Why we cannot delete



- Clearing Ahmed's bits also clears bit 4, which Sara needs.
- Sara is now reported "definitely not": a false negative!
- The fix, a counting Bloom filter: a small counter per cell instead of a bit, incremented by `add` and decremented by `remove`.



Outline

1 The problem: searching by prefix
2 Tries
3 Operations on a trie
4 Cost of a trie
5 Tries in practice
6 Probabilistic data structures
7 Bloom filters
8 How wrong can a Bloom filter be?
9 Bloom filters in practice
10 Summary



The false positive rate

n elements added, m bits, k hash functions (assumed uniform and independent).

- One hash sets a given bit with probability $1/m$.



The false positive rate

n elements added, m bits, k hash functions (assumed uniform and independent).

- One hash sets a given bit with probability $1/m$.
- After n insertions (kn hashes), a bit is still 0 with probability $\left(1 - \frac{1}{m}\right)^{kn} \approx e^{-kn/m}$.



The false positive rate

n elements added, m bits, k hash functions (assumed uniform and independent).

- One hash sets a given bit with probability $1/m$.
- After n insertions (kn hashes), a bit is still 0 with probability $\left(1 - \frac{1}{m}\right)^{kn} \approx e^{-kn/m}$.
- A false positive: the k bits of an absent element are all 1:



The false positive rate

n elements added, m bits, k hash functions (assumed uniform and independent).

- One hash sets a given bit with probability $1/m$.
- After n insertions (kn hashes), a bit is still 0 with probability $\left(1 - \frac{1}{m}\right)^{kn} \approx e^{-kn/m}$.
- A false positive: the k bits of an absent element are all 1:

$$p \approx \left(1 - e^{-kn/m}\right)^k$$



The false positive rate

n elements added, m bits, k hash functions (assumed uniform and independent).

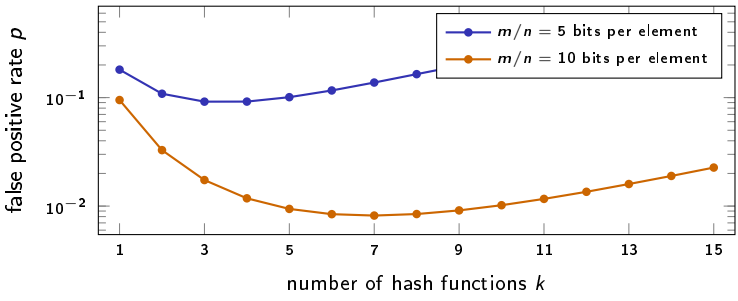
- One hash sets a given bit with probability $1/m$.
- After n insertions (kn hashes), a bit is still 0 with probability $\left(1 - \frac{1}{m}\right)^{kn} \approx e^{-kn/m}$.
- A false positive: the k bits of an absent element are all 1:

$$p \approx \left(1 - e^{-kn/m}\right)^k$$

- p depends only on m/n (bits per element) and k.



Choosing k



- Too few hashes: each check is weak. Too many: the array fills with 1s.
- Best choice: $k = \frac{m}{n} \ln 2 \approx 0.7 \frac{m}{n}$. Then half of the bits are 1.

Sizing a filter

With the best k:

Bits per element m/n	Best k	False positive rate p
5	3	9.2%
8	6	2.2%
10	7	0.82%
15	10	0.074%
20	14	0.0067%



Sizing a filter

With the best k:

Bits per element m/n	Best k	False positive rate p
5	3	9.2%
8	6	2.2%
10	7	0.82%
15	10	0.074%
20	14	0.0067%

- About 10 bits per element for 1%; 5 more bits divide p by 10.
- The memory does not depend on the size of the elements.



Back to the malicious URLs

1 million URLs, false positive rate 1%:

Structure	Memory	Answers
`HashSet`	$\approx$ 100 MB	exact
Bloom filter	$\approx$ 1.2 MB	"no" exact, "maybe" at 1%



Back to the malicious URLs

1 million URLs, false positive rate 1%:

Structure	Memory	Answers
`HashSet`	$\approx$ 100 MB	exact
Bloom filter	$\approx$ 1.2 MB	"no" exact, "maybe" at 1%

- 80 times less memory: the filter fits in any browser.
- 1 visit in 100 to a safe site needs an extra check with the server.

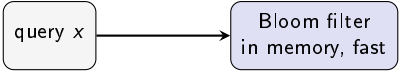


Outline

1 The problem: searching by prefix
2 Tries
3 Operations on a trie
4 Cost of a trie
5 Tries in practice
6 Probabilistic data structures
7 Bloom filters
8 How wrong can a Bloom filter be?
9 Bloom filters in practice
10 Summary

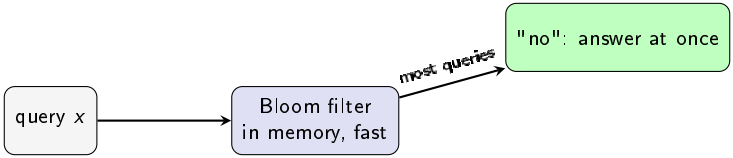


A cheap filter in front of an expensive check



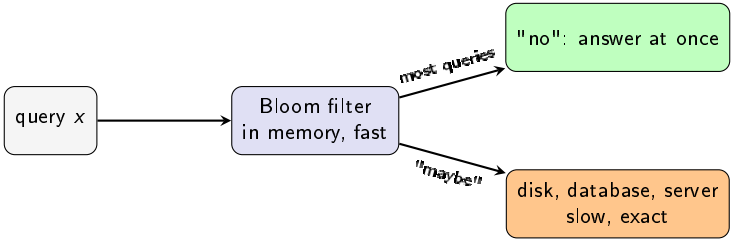


A cheap filter in front of an expensive check

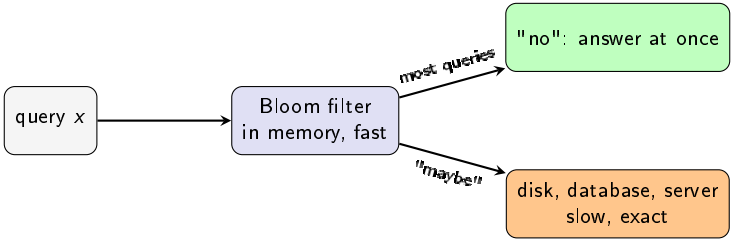




A cheap filter in front of an expensive check



A cheap filter in front of an expensive check



- The filter never changes the final answer: it only avoids useless slow checks.
- A false positive costs one extra check, never a wrong result.



Where are Bloom filters used?

- Databases (Cassandra, HBase, RocksDB): one filter per file on disk, to skip the files that cannot contain a key.
- Browsers: checking URLs against a list of malicious sites.
- Web caches: cache a page only at its second request; most pages are requested once.
- Spell checkers and password checkers: is this word in a huge list?



In the libraries

No Bloom filter in the Java, Python or C++ standard libraries.

```
// Google Guava
BloomFilter<String> urls = BloomFilter.create(
    Funnels.stringFunnel(StandardCharsets.UTF_8),
    1_000_000,  // expected number of elements
    0.01);      // false positive rate

urls.put("http://malicious.example");
urls.mightContain("http://usto.dz"); // false (or, rarely, true)
```

- Python: packages on PyPI, such as `pybloom-live`.
- Redis: `BF.ADD` and `BF.EXISTS` commands.



Outline

1 The problem: searching by prefix
2 Tries
3 Operations on a trie
4 Cost of a trie
5 Tries in practice
6 Probabilistic data structures
7 Bloom filters
8 How wrong can a Bloom filter be?
9 Bloom filters in practice
10 Summary



Key takeaways

- A trie stores words as paths of letters: search, insert and prefix queries in $O(m)$, independent of n.
- Probabilistic structures trade a small, controlled error for much less memory.
- A Bloom filter: k hash functions, m bits. "No" is certain, "yes" means "maybe".
- About 10 bits per element give 1% of false positives, whatever the size of the elements.



From data structures to algorithms

- Hash tables, dynamic arrays, tries, Bloom filters: the choice of the data structure decides the efficiency of a program, in time and in memory.



From data structures to algorithms

- Hash tables, dynamic arrays, tries, Bloom filters: the choice of the data structure decides the efficiency of a program, in time and in memory.
- But a good structure is not enough: the algorithm that uses it matters just as much.



From data structures to algorithms

- Hash tables, dynamic arrays, tries, Bloom filters: the choice of the data structure decides the efficiency of a program, in time and in memory.
- But a good structure is not enough: the algorithm that uses it matters just as much.
- Next part, algorithmic paradigms: general methods to design efficient algorithms (brute force, divide and conquer, greedy algorithms, dynamic programming, ...).

